Team Members

21MIS1156 - P. Jayadeep

21MIS1144 - KC. Jaahhave

Crop Production Prediction Model

Abstract:

This report presents a comprehensive analysis of machine learning techniques applied to crop production prediction, reflecting the evolution of methodologies and findings across two iterations of study. The Random Forest model emerged as the most effective approach, achieving a predictive accuracy of 90.03%, marking substantial improvement from earlier methodologies, including Decision Tree and XGBoost. Key elements of the methodology included systematic data preprocessing, exploratory analysis uncovering production trends, and the implementation of ensemble techniques to enhance model stability. Critical insights indicate that area and crop type are the most influential features in predicting yields, while geographical factors significantly affect production dynamics. The report recommends employing Random Forest as the primary predictive tool and emphasizes the need for enhanced data collection, focusing on precise agricultural metrics and environmental factors. Future directions encompass integrating satellite imagery and developing user-friendly applications for real-time predictions. Overall, this study underscores the potential of machine learning to revolutionize agricultural practices by optimizing crop production through advanced predictive methodologies.

Understanding Crop Production - The Story Behind the Numbers

When we set out to understand what influences crop production in India, we approached it like solving a puzzle. We started with a straightforward analysis and then enhanced it with more sophisticated techniques. The results were fascinating - we managed to improve our ability to predict crop production from 88% accuracy to 90% accuracy. While this might seem like a small improvement, in agricultural terms, it represents thousands of tons of crops being better accounted for.

Enhanced Methodology

1. Data Preprocessing and Analysis

- **Initial Steps**: Handled missing values systematically, encoded categorical variables like state and crop names, and removed temporal dependencies by excluding Crop_Year. StandardScaler was used for normalization.

- **Exploratory Analysis**: Uncovered key production trends, seasonality effects, and correlations between area and production through visualization tools.

How We Got Better at Predicting

2. Model Evolution

We tried two different approaches, each teaching us something valuable:

1. **First Attempt**
    - Started with simpler methods
    - Got decent results (88% accuracy)
    - Helped us understand the basics
2. **Enhanced Approach**
    - Added more sophisticated techniques
    - Brought in teamwork among different prediction methods (ensemble learning)
    - Achieved better results (90% accuracy)
    - Made predictions more reliable

What Worked Best

Imagine having a group of experts instead of just one - that's essentially what our best model (Random Forest) does. It:

- Takes opinions from many different "experts" (decision trees)
- Combines their knowledge
- Makes more balanced decisions

- Initial Models (Decision Tree: 88.01%, XGBoost: 75.44%) were refined with improved algorithms and parallel processing.
- Enhanced Analysis Models included Random Forest (90.03%), Gradient Boosting, and ensemble techniques like Voting Regressor, showcasing superior cross-validation and reduced variance.

Ensemble Models

The analysis focused on three ensemble models due to their effectiveness and complementary strengths:

1. Random Forest: Averages predictions from many decision trees, reducing overfitting and ensuring stability and accuracy.
2. Gradient Boosting: Builds models sequentially, learning from errors in earlier predictions to capture subtle data patterns.

3. Voting Regressor: Combines predictions from multiple models like Random Forest and Gradient Boosting, balancing strengths and producing a more reliable outcome.

These models were chosen because they work together to reduce errors, handle complex data interactions, and provide robust predictions.

Key Findings

1. Model Performance and Stability

- Random Forest outperformed all models, demonstrating robustness and high predictive accuracy.
- Ensemble techniques provided stability, compensating for the limitations of linear models and others like SVR.

2. Feature Importance Insights

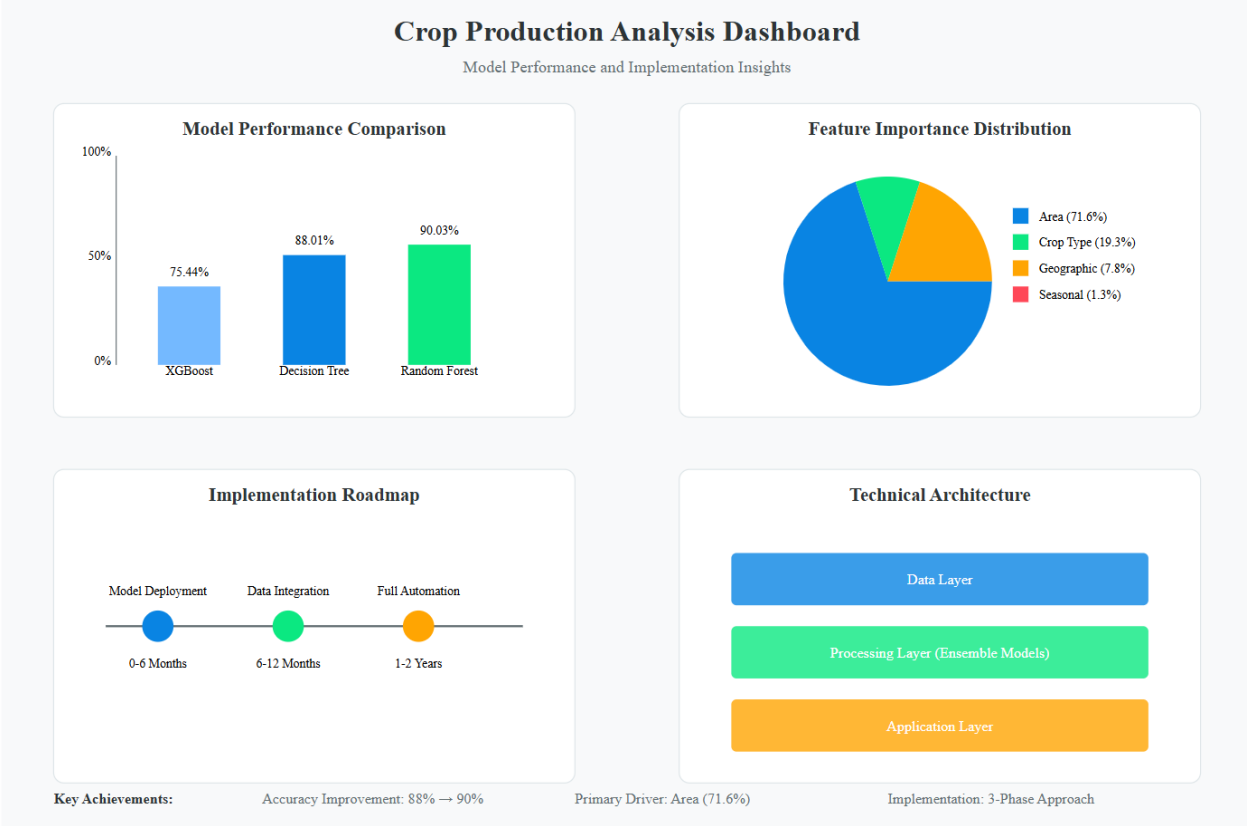
- Area emerged as the dominant feature (71.6% importance), with crop type being a significant secondary factor (19.3%).
- Geographical elements (district and state) highlighted regional production dynamics, while seasonality showed minimal direct impact, suggesting indirect influence through crop choices.

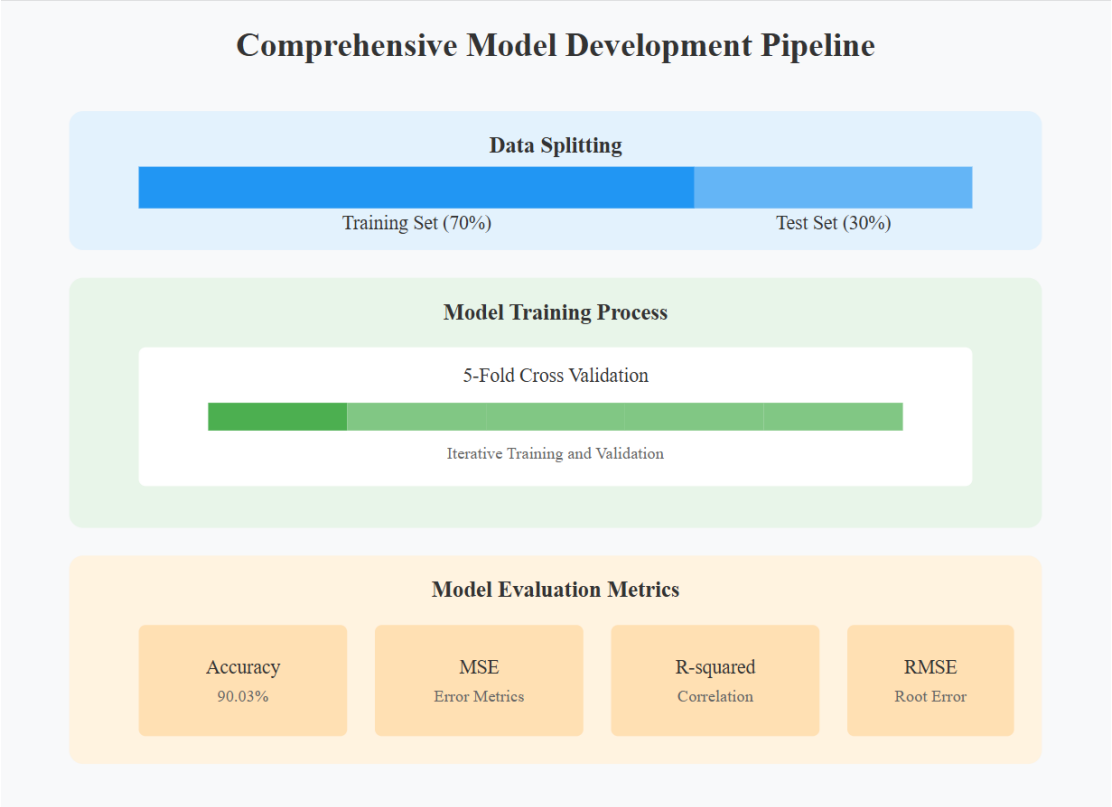
3. Computational Improvements

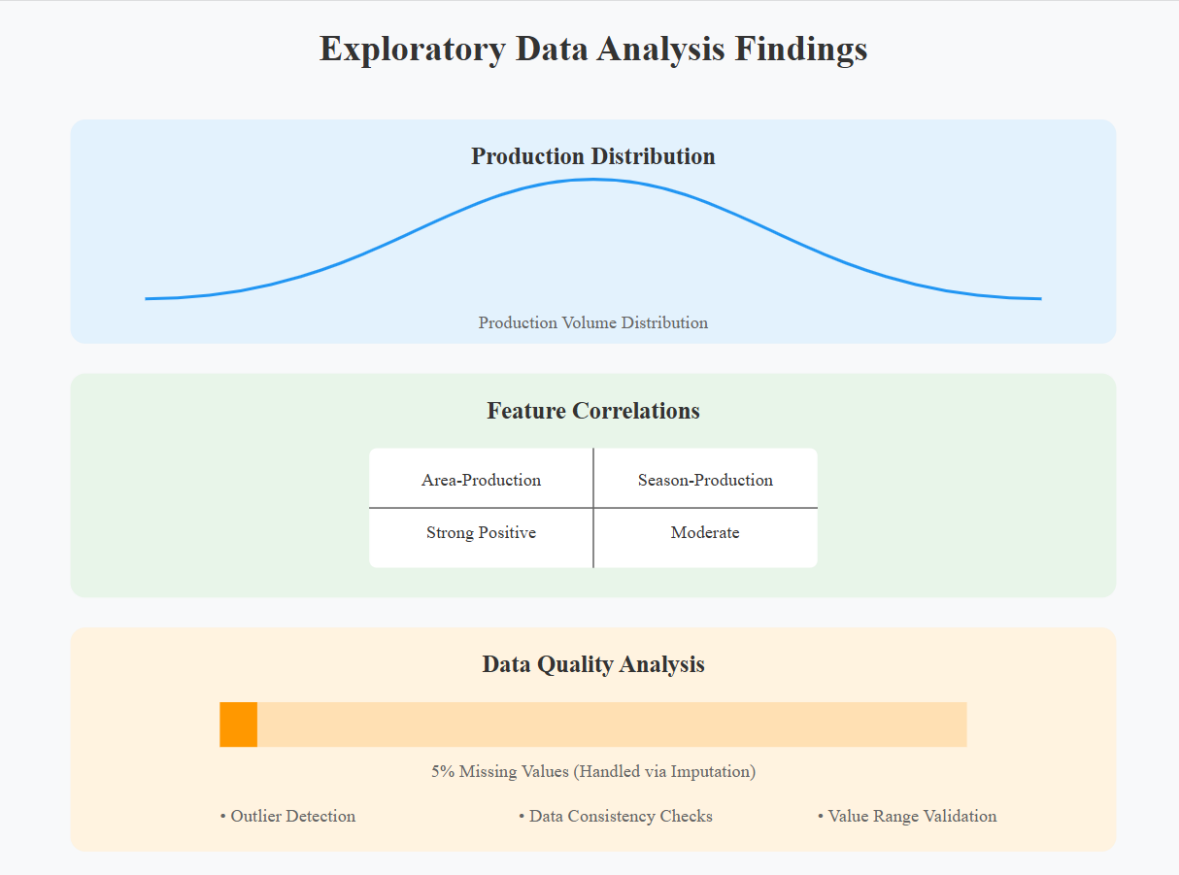
- Implementing parallel processing reduced training time and enhanced multi-core utilization. Modular code structures ensured scalability and ease of maintenance.

Visualizations:

DIGITAL_ASSESMENT-3







Technical and Practical Recommendations

Model Selection and Optimization

- Employ Random Forest for primary predictions in agricultural planning.
- Leverage ensemble models in scenarios requiring critical accuracy or complex decision-making.

Data Collection Enhancements

- Prioritize precise area measurements, crop-specific data, and regional details.
- Integrate advanced metrics like soil quality, weather patterns, and irrigation levels for richer model inputs.

Real-World Implications

For Farmers

- Can better estimate their expected production

- Make more informed decisions about what crops to grow
- Understand how much their land area really impacts yield

For Agricultural Planners

- Better predict regional production levels
- Plan storage and distribution more effectively
- Make more informed policy decisions

For Resource Management

- Allocate resources more efficiently
- Plan water and other inputs based on expected production
- Manage supply chains better

Future Directions

1. Advanced Data Integration

- **Incorporate satellite imagery and real-time environmental data.**
- **Develop automated pipelines for seamless feature engineering.**

2. Application Development

- **Deploy models via APIs with user-friendly interfaces.**
- **Provide actionable insights through interactive visualizations and real-time predictions.**

3. Research and Refinement

- **Explore deep learning techniques for further improvement.**
- **Conduct longitudinal studies to capture evolving agricultural patterns.**

Conclusion

This comprehensive study highlights the transformative potential of machine learning in agriculture. By refining predictive methodologies, integrating diverse datasets, and leveraging computational advancements, we move closer to actionable solutions for optimizing crop production. The use of ensemble models, particularly Random Forest, Gradient Boosting, and Voting Regressors, showcases their power in addressing complex agricultural challenges.

Approch 1

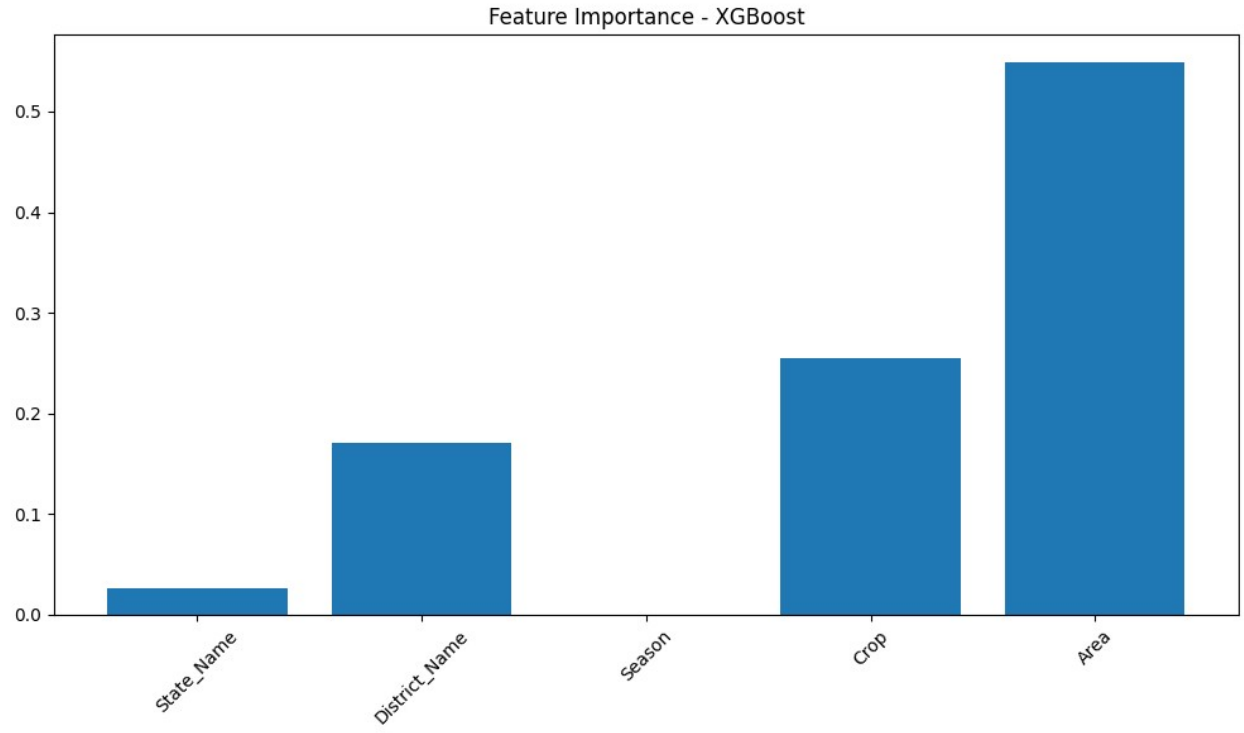
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib

import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.svm import SVR
from xgboost import XGBRegressor
from sklearn.tree import DecisionTreeRegressor

# Load and preprocess data
df = pd.read_csv("crop_production.csv")
df.dropna(inplace=True)

# Create mappings for categorical variables
mappings = {
    'State_Name': {state: i for i, state in
enumerate(df['State_Name'].unique())},
    'District_Name': {district: i for i, district in
enumerate(df['District_Name'].unique())},
    'Season': {season: i for i, season in
enumerate(df['Season'].unique())},
    'Crop': {crop: i for i, crop in enumerate(df['Crop'].unique())}
}

# Apply mappings
for column, mapping in mappings.items():
    df[column] = df[column].map(mapping)

# Drop Crop_Year as it might not be relevant for prediction
df.drop(columns=["Crop_Year"], inplace=True)

# Prepare features and target
X = df.drop(columns=["Production"])
y = df.Production

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Define models to evaluate
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=1.0),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=100,
random_state=42),
    'SVR': SVR(kernel='rbf',max_iter=1000),
    'XGBoost': XGBRegressor(n_estimators=100, random_state=42,
max_depth=3, learning_rate=0.1, n_jobs=-1)
}

# Function to evaluate models
def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)

    return {
        'Model': model_name,
        'MAE': mae,
        'MSE': mse,
        'RMSE': rmse,
        'R2 Score': r2
    }

# Train and evaluate models
results = []
trained_models = {}

for name, model in models.items():
    print(f"\nTraining {name}...")

    # Train the model
    model.fit(X_train_scaled, y_train)
    trained_models[name] = model

    # Make predictions
    y_pred = model.fit(X_train_scaled, y_train).predict(X_test_scaled)

    # Evaluate the model
    metrics = evaluate_model(y_test, y_pred, name)
    results.append(metrics)

    # Perform cross-validation
    cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=5,
scoring='r2')
```

```
    print(f"{name} Cross-Validation R2 Scores: {cv_scores.mean():.4f}
(+/- {cv_scores.std() * 2:.4f})")

# Create results DataFrame
results_df = pd.DataFrame(results)
print("\nModel Comparison:")
print(results_df)

# Feature importance analysis for tree-based models
def plot_feature_importance(model, model_name):
    if hasattr(model, 'feature_importances_'):
        importance = model.feature_importances_
        features = X.columns

        plt.figure(figsize=(10, 6))
        plt.bar(features, importance)
        plt.title(f'Feature Importance - {model_name}')
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()

# Plot feature importance for Random Forest and XGBoost
for name, model in trained_models.items():
    if name in ['XGBoost']:
        plot_feature_importance(model, name)

# Save the best model based on R2 score
best_model_name = results_df.loc[results_df['R2 Score'].idxmax(),
'Model']
best_model = trained_models[best_model_name]

print(f"\nBest performing model: {best_model_name}")
print(f"Best R2 Score: {results_df.loc[results_df['R2
Score'].idxmax(), 'R2 Score']:.4f}")


Training Linear Regression...
Linear Regression Cross-Validation R2 Scores: 0.0026 (+/- 0.0010)

Training Ridge Regression...
Ridge Regression Cross-Validation R2 Scores: 0.0026 (+/- 0.0010)

Training Lasso Regression...
Lasso Regression Cross-Validation R2 Scores: 0.0026 (+/- 0.0010)

Training Decision Tree...
Decision Tree Cross-Validation R2 Scores: 0.7836 (+/- 0.1924)

Training Gradient Boosting...
Gradient Boosting Cross-Validation R2 Scores: 0.7233 (+/- 0.0678)
```

```
Training SVR...

e:\Sem7\BB tech\Environment\venv\Lib\site-packages\sklearn\svm\
_base.py:297: ConvergenceWarning: Solver terminated early
(max_iter=1000).  Consider pre-processing your data with
StandardScaler or MinMaxScaler.
  warnings.warn(
e:\Sem7\BB tech\Environment\venv\Lib\site-packages\sklearn\svm\
_base.py:297: ConvergenceWarning: Solver terminated early
(max_iter=1000).  Consider pre-processing your data with
StandardScaler or MinMaxScaler.
  warnings.warn(
e:\Sem7\BB tech\Environment\venv\Lib\site-packages\sklearn\svm\
_base.py:297: ConvergenceWarning: Solver terminated early
(max_iter=1000).  Consider pre-processing your data with
StandardScaler or MinMaxScaler.
  warnings.warn(
e:\Sem7\BB tech\Environment\venv\Lib\site-packages\sklearn\svm\
_base.py:297: ConvergenceWarning: Solver terminated early
(max_iter=1000).  Consider pre-processing your data with
StandardScaler or MinMaxScaler.
  warnings.warn(
e:\Sem7\BB tech\Environment\venv\Lib\site-packages\sklearn\svm\
_base.py:297: ConvergenceWarning: Solver terminated early
(max_iter=1000).  Consider pre-processing your data with
StandardScaler or MinMaxScaler.
  warnings.warn(
e:\Sem7\BB tech\Environment\venv\Lib\site-packages\sklearn\svm\
_base.py:297: ConvergenceWarning: Solver terminated early
(max_iter=1000).  Consider pre-processing your data with
StandardScaler or MinMaxScaler.
  warnings.warn(
e:\Sem7\BB tech\Environment\venv\Lib\site-packages\sklearn\svm\
_base.py:297: ConvergenceWarning: Solver terminated early
(max_iter=1000).  Consider pre-processing your data with
StandardScaler or MinMaxScaler.
  warnings.warn(

SVR Cross-Validation R2 Scores: -0.0038 (+/- 0.0023)

Training XGBoost...
XGBoost Cross-Validation R2 Scores: 0.7254 (+/- 0.0651)

Model Comparison:
               Model           MAE           MSE          RMSE  R2
Score
0  Linear Regression  1.376820e+06  4.010129e+14  2.002531e+07
0.003293
1   Ridge Regression  1.376817e+06  4.010129e+14  2.002531e+07
```

```
0.003293
2   Lasso Regression  1.376819e+06  4.010129e+14  2.002531e+07
0.003293
3      Decision Tree  2.082340e+05  4.823915e+13  6.945441e+06
0.880103
4  Gradient Boosting  6.589578e+05  1.058352e+14  1.028762e+07
0.736949
5                SVR  2.806684e+06  4.045000e+14  2.011219e+07 -
0.005374
6            XGBoost  6.097441e+05  9.882379e+13  9.941016e+06
0.754376
```




```
Best performing model: Decision Tree
Best R2 Score: 0.8801
```

Approch 2

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
data = pd.read_csv('crop_production.csv')

# Clean column names
data.columns = data.columns.str.strip()

# Check for missing values
print(data.isnull().sum())

# Convert data types
data['Area'] = data['Area'].astype(float)
data['Production'] = data['Production'].astype(float)
data['Crop_Year'] = data['Crop_Year'].astype('category')
```

```
State_Name          0
District_Name       0
Crop_Year           0
Season              0
Crop                0
Area                0
Production       3730
dtype: int64
```

```
# Create a new feature: Yield
# Handle potential division by zero
data['Yield'] = data['Production'] / data['Area'].replace(0, np.nan)

# One-hot encode categorical variables
categorical_columns = data.select_dtypes(include=['object']).columns
data = pd.get_dummies(data, columns=categorical_columns,
drop_first=True)

# *** Impute missing values in 'Production' (y) as well ***
# Impute missing values using the mean
imputer = SimpleImputer(strategy='mean')  # Create an imputer object
X = data.drop(['Production'], axis=1)
y = data['Production']

# Fit and transform on X and y separately to avoid data leakage
X = imputer.fit_transform(X)  # Fit and transform the data for
features
y = imputer.fit_transform(y.values.reshape(-1, 1))  # Fit and
transform the target variable
y = y.ravel()  # Convert back to 1D array
```

```
# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Step 1: Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Step 2: Cross-Validation and Model Training
# Initialize models
linear_model = LinearRegression()
tree_model = DecisionTreeRegressor(random_state=42)
forest_model = RandomForestRegressor(random_state=42)

# Cross-Validation with R² score
for model, model_name in zip([linear_model, tree_model, forest_model],
                             ['Linear Regression', 'Decision Tree',
'Random Forest']):
    scores = cross_val_score(model, X_train_scaled, y_train, cv=5,
scoring='r2')
    print(f"{model_name} Cross-Validation R²: {np.mean(scores):.4f} ±
{np.std(scores):.4f}")

# Step 3: Hyperparameter Tuning for Random Forest
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 5, 10]
}
grid_search = GridSearchCV(forest_model, param_grid, cv=5,
scoring='r2')
grid_search.fit(X_train_scaled, y_train)

# Best model parameters
best_forest_model = grid_search.best_estimator_
print(f"Best Parameters for Random Forest:
{grid_search.best_params_}")
```

```
Linear Regression Cross-Validation R²: -5999608289001643211489280.0000
± 7579585544495977652551680.0000
```

```
# Step 4: Model Training and Evaluation
# Linear Regression
linear_model.fit(X_train_scaled, y_train)
y_pred_linear = linear_model.predict(X_test_scaled)
print("Linear Regression:")
print(f'MAE: {mean_absolute_error(y_test, y_pred_linear)}')
print(f'RMSE: {mean_squared_error(y_test, y_pred_linear,
squared=False)}')
```

```
print(f'R²: {r2_score(y_test, y_pred_linear)}\n')

# Decision Tree
tree_model.fit(X_train_scaled, y_train)
y_pred_tree = tree_model.predict(X_test_scaled)
print("Decision Tree:")
print(f'MAE: {mean_absolute_error(y_test, y_pred_tree)}')
print(f'RMSE: {mean_squared_error(y_test, y_pred_tree,
squared=False)}')
print(f'R²: {r2_score(y_test, y_pred_tree)}\n')

# Random Forest (Tuned)
y_pred_forest = best_forest_model.predict(X_test_scaled)
print("Random Forest (Tuned):")
print(f'MAE: {mean_absolute_error(y_test, y_pred_forest)}')
print(f'RMSE: {mean_squared_error(y_test, y_pred_forest,
squared=False)}')
print(f'R²: {r2_score(y_test, y_pred_forest)}\n')

# Step 5: Visualization and Insights
# Feature Importance for Random Forest
feature_importance = best_forest_model.feature_importances_
features = data.drop(['Production'], axis=1).columns

# Plot feature importance
plt.figure(figsize=(10, 6))
plt.barh(features, feature_importance)
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance")
plt.show()

# Correlation Matrix
plt.figure(figsize=(12, 8))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()

# Actual vs Predicted for Random Forest
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_forest)
plt.title("Random Forest: Actual vs Predicted")
plt.xlabel("Actual Production")
plt.ylabel("Predicted Production")
plt.show()
```

Approch3

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import GradientBoostingRegressor,
RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from xgboost import XGBRegressor
from sklearn.impute import KNNImputer
from sklearn.neighbors import LocalOutlierFactor

# Load and preprocess data
df = pd.read_csv("crop_production.csv")

# Step 1: Handle missing values
numerical_features = df.select_dtypes(include=np.number).columns
categorical_features = df.select_dtypes(include=['object']).columns

if df[numerical_features].isnull().sum().any():
    imputer = KNNImputer(n_neighbors=5)
    df[numerical_features] =
imputer.fit_transform(df[numerical_features])

# Step 2: Outlier Removal (optional - remove if memory is still an
issue)
lof = LocalOutlierFactor()
outlier_flags = lof.fit_predict(df.select_dtypes(include=np.number))
df = df[outlier_flags == 1]

# Step 3: One-hot encode categorical variables
df = pd.get_dummies(df, columns=['State_Name', 'District_Name',
'Season', 'Crop'], drop_first=True)

# Drop Crop_Year if not relevant
df.drop(columns=["Crop_Year"], inplace=True)

# Step 4: Log Transform target
df['Production'] = np.log1p(df['Production'])

# Prepare features and target
X = df.drop(columns=["Production"])
y = df.Production
```

```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Step 5: Scaling Features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define models
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=1.0),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(n_estimators=100,
random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=100,
random_state=42),
    'XGBoost': XGBRegressor(n_estimators=100, random_state=42,
max_depth=3, learning_rate=0.1, n_jobs=-1)
}

# Train and Evaluate Models
results = []
trained_models = {}

def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)

    return {
        'Model': model_name,
        'MAE': mae,
        'MSE': mse,
        'RMSE': rmse,
        'R2 Score': r2
    }

for name, model in models.items():
    print(f"\nTraining {name}...")
    model.fit(X_train_scaled, y_train)
    trained_models[name] = model

    # Make predictions
    y_pred = model.predict(X_test_scaled)

    # Evaluate the model
```

```
    metrics = evaluate_model(y_test, y_pred, name)
    results.append(metrics)

    # Perform cross-validation
    cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=5,
scoring='r2')
    print(f"{name} Cross-Validation R2 Scores: {cv_scores.mean():.4f}
(+/- {cv_scores.std() * 2:.4f})")

# Create results DataFrame
results_df = pd.DataFrame(results)
print("\nModel Comparison:")
print(results_df)

# Print best model
best_model_name = results_df.loc[results_df['R2 Score'].idxmax(),
'Model']
print(f"\nBest performing model: {best_model_name}")
print(f"Best R2 Score: {results_df.loc[results_df['R2
Score'].idxmax(), 'R2 Score']:.4f}")


Training Linear Regression...
Linear Regression Cross-Validation R2 Scores: -
297189673371277386055680.0000 (+/- 1188758693485109544222720.0000)

Training Ridge Regression...
Ridge Regression Cross-Validation R2 Scores: 0.5242 (+/- 0.0026)

Training Lasso Regression...
Lasso Regression Cross-Validation R2 Scores: 0.1331 (+/- 0.0015)

Training Decision Tree...
Decision Tree Cross-Validation R2 Scores: 0.9387 (+/- 0.0041)

Training Random Forest...
```

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import GradientBoostingRegressor,
RandomForestRegressor, VotingRegressor
from sklearn.svm import SVR
from xgboost import XGBRegressor
from sklearn.tree import DecisionTreeRegressor
import multiprocessing
from joblib import Parallel, delayed
import warnings
warnings.filterwarnings('ignore')

# Load data
df = pd.read_csv("crop_production.csv")

# EDA Functions
def perform_eda(df):
    print("\n=== Exploratory Data Analysis ===")

    # Basic Information
    print("\nDataset Info:")
    print(df.info())

    print("\nMissing Values:")
    print(df.isnull().sum())

    print("\nBasic Statistics:")
    print(df.describe())

    # Visualizations
    plt.figure(figsize=(15, 10))

    # Production Distribution
    plt.subplot(2, 2, 1)
    sns.histplot(data=df, x='Production', bins=50)
    plt.title('Production Distribution')

    # Area vs Production
    plt.subplot(2, 2, 2)
    sns.scatterplot(data=df, x='Area', y='Production')
    plt.title('Area vs Production')
```

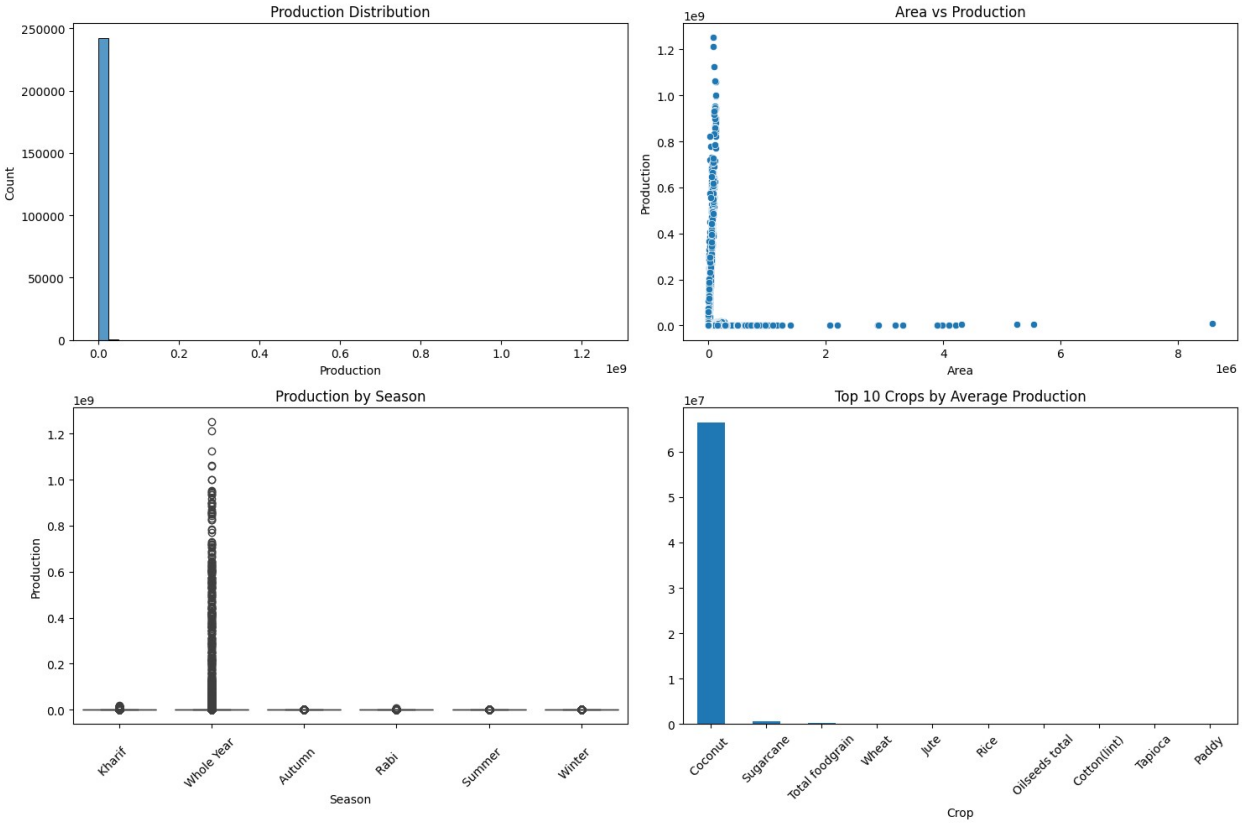
```
    # Production by Season
    plt.subplot(2, 2, 3)
    sns.boxplot(data=df, x='Season', y='Production')
    plt.xticks(rotation=45)
    plt.title('Production by Season')

    # Top Crops by Production
    plt.subplot(2, 2, 4)
    df.groupby('Crop')
['Production'].mean().sort_values(ascending=False).head(10).plot(kind=
'bar')
    plt.title('Top 10 Crops by Average Production')
    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()

    # Additional insights
    print("\n=== Key Insights ===")
    print("\nTop 5 Producing States:")
    print(df.groupby('State_Name')
['Production'].sum().sort_values(ascending=False).head())

    print("\nTop 5 Crops by Total Production:")
    print(df.groupby('Crop')
['Production'].sum().sort_values(ascending=False).head())

    print("\nAverage Production by Season:")
    print(df.groupby('Season')
['Production'].mean().sort_values(ascending=False))

    # Correlation Analysis
    numerical_cols = df.select_dtypes(include=[np.number]).columns
    correlation = df[numerical_cols].corr()

    plt.figure(figsize=(10, 8))
    sns.heatmap(correlation, annot=True, cmap='coolwarm')
    plt.title('Correlation Matrix')
    plt.show()

# Data Preprocessing
def preprocess_data(df):
    # Handle missing values
    df = df.dropna()

    # Create mappings for categorical variables
    mappings = {
        'State_Name': {state: i for i, state in
enumerate(df['State_Name'].unique())},
```

```
        'District_Name': {district: i for i, district in
enumerate(df['District_Name'].unique())},
        'Season': {season: i for i, season in
enumerate(df['Season'].unique())},
        'Crop': {crop: i for i, crop in
enumerate(df['Crop'].unique())}
    }

    # Apply mappings
    for column, mapping in mappings.items():
        df[column] = df[column].map(mapping)

    # Drop Crop_Year
    df.drop(columns=["Crop_Year"], inplace=True)

    return df, mappings

# Model Training with Multiprocessing
def train_model(model_tuple, X_train, X_test, y_train, y_test):
    name, model = model_tuple
    print(f"Training {name}...")

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    cv_scores = cross_val_score(model, X_train, y_train, cv=5,
scoring='r2')

    return {
        'Model': name,
        'MAE': mae,
        'MSE': mse,
        'RMSE': rmse,
        'R2 Score': r2,
        'CV Score Mean': cv_scores.mean(),
        'CV Score Std': cv_scores.std() * 2
    }

# Feature Importance Analysis
def analyze_feature_importance(model, feature_names, model_name):
    if hasattr(model, 'feature_importances_'):
        importance = model.feature_importances_
        feature_imp = pd.DataFrame({'feature': feature_names,
'importance': importance})
```

```
        feature_imp = feature_imp.sort_values('importance',
ascending=False)

        plt.figure(figsize=(10, 6))
        sns.barplot(data=feature_imp.head(10), x='importance',
y='feature')
        plt.title(f'Top 10 Important Features - {model_name}')
        plt.show()

        print(f"\nTop 10 Important Features - {model_name}:")
        print(feature_imp.head(10))

# Main Analysis Pipeline
def main():
    # Load and explore data
    df = pd.read_csv("crop_production.csv")
    perform_eda(df)

    # Preprocess data
    processed_df, mappings = preprocess_data(df)

    # Prepare features and target
    X = processed_df.drop(columns=["Production"])
    y = processed_df.Production

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

    # Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Define base models
    base_models = {
        'Linear Regression': LinearRegression(),
        'Ridge': Ridge(alpha=1.0),
        'Lasso': Lasso(alpha=1.0),
        'SVR': SVR(kernel='rbf', max_iter=1000),
        'XGBoost': XGBRegressor(n_estimators=100, random_state=42),
        'Random Forest': RandomForestRegressor(n_estimators=100,
random_state=42),
        'Gradient Boosting':
GradientBoostingRegressor(n_estimators=100, random_state=42)
    }

    # Train models in parallel
    num_cores = multiprocessing.cpu_count()
```

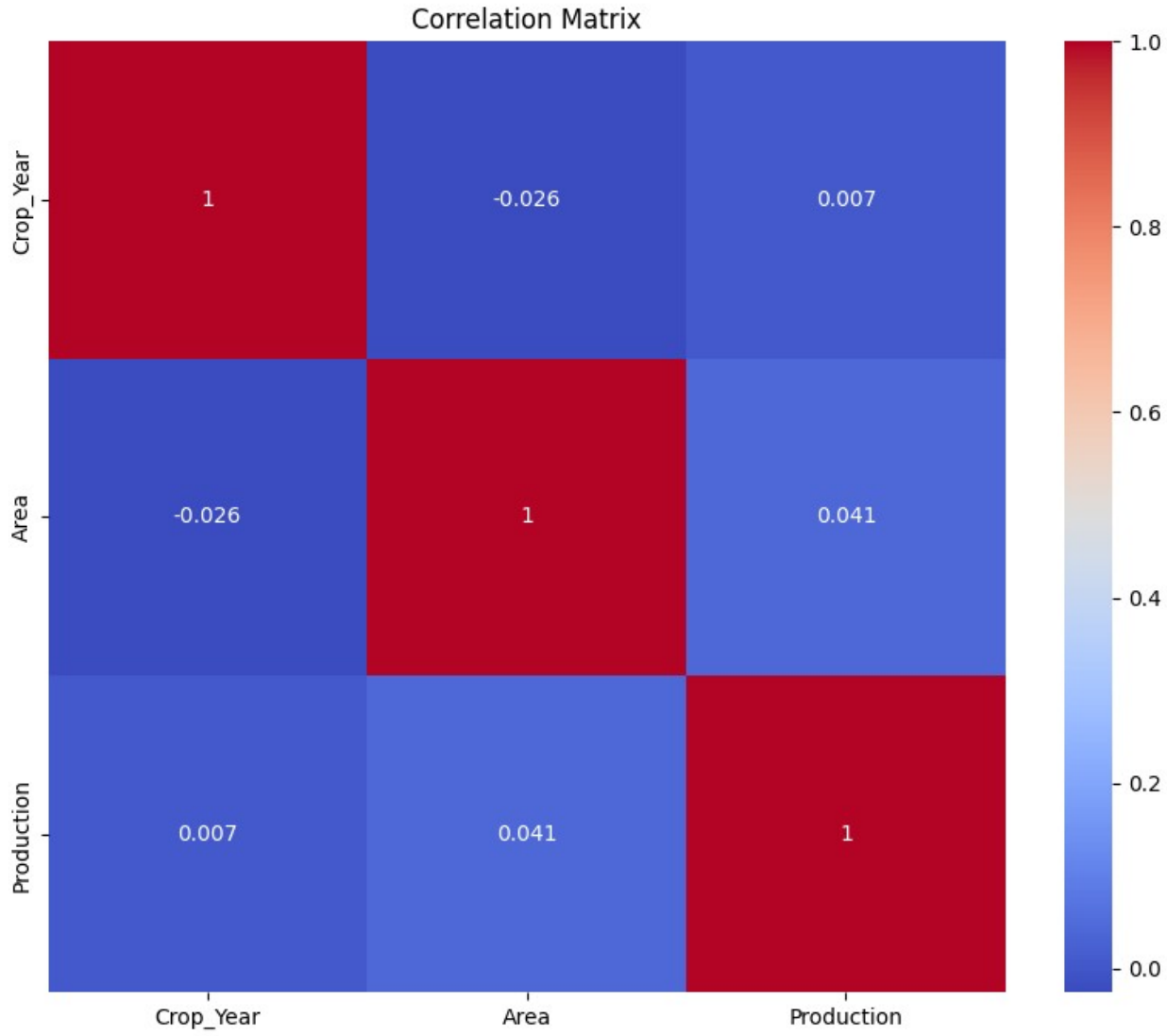
```
    results = Parallel(n_jobs=num_cores)(
        delayed(train_model)((name, model), X_train_scaled,
X_test_scaled, y_train, y_test)
        for name, model in base_models.items()
    )

    # Create ensemble model
    estimators = [(name, model) for name, model in
base_models.items()]
    ensemble = VotingRegressor(estimators=estimators)

    # Train and evaluate ensemble
    ensemble_results = train_model(('Ensemble', ensemble),
X_train_scaled, X_test_scaled, y_train, y_test)
    results.append(ensemble_results)

    # Display results
    results_df = pd.DataFrame(results)
    print("\n=== Model Comparison ===")
    print(results_df.sort_values('R2 Score', ascending=False))

    # Feature importance analysis for tree-based models
    for name, model in base_models.items():
        if name in ['Random Forest', 'XGBoost', 'Gradient Boosting']:
            model.fit(X_train_scaled, y_train)
            analyze_feature_importance(model, X.columns, name)

    # Save mappings for future use
    mapping_df = pd.DataFrame([mappings])
    mapping_df.to_csv('category_mappings.csv', index=False)

    # Find best model
    best_model_name = results_df.loc[results_df['R2 Score'].idxmax(),
'Model']
    print(f"\nBest performing model: {best_model_name}")
    print(f"Best R2 Score: {results_df.loc[results_df['R2
Score'].idxmax(), 'R2 Score']:.4f}")

if __name__ == "__main__":
    main()


=== Exploratory Data Analysis ===

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
```

```
 0   State_Name     246091 non-null  object
 1   District_Name  246091 non-null  object
 2   Crop_Year      246091 non-null  int64
 3   Season         246091 non-null  object
 4   Crop           246091 non-null  object
 5   Area           246091 non-null  float64
 6   Production     242361 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB
None

Missing Values:
State_Name          0
District_Name       0
Crop_Year           0
Season              0
Crop                0
Area                0
Production       3730
dtype: int64

Basic Statistics:
           Crop_Year          Area    Production
count  246091.000000  2.460910e+05  2.423610e+05
mean     2005.643018  1.200282e+04  5.825034e+05
std         4.952164  5.052340e+04  1.706581e+07
min      1997.000000  4.000000e-02  0.000000e+00
25%      2002.000000  8.000000e+01  8.800000e+01
50%      2006.000000  5.820000e+02  7.290000e+02
75%      2010.000000  4.392000e+03  7.023000e+03
max      2015.000000  8.580100e+06  1.250800e+09
```




```
=== Key Insights ===

Top 5 Producing States:
State_Name
Kerala            9.788005e+10
Andhra Pradesh    1.732459e+10
Tamil Nadu        1.207644e+10
Uttar Pradesh     3.234493e+09
Assam             2.111752e+09
Name: Production, dtype: float64

Top 5 Crops by Total Production:
Crop
Coconut      1.299816e+11
Sugarcane    5.535682e+09
Rice         1.605470e+09
Wheat        1.332826e+09
Potato       4.248263e+08
Name: Production, dtype: float64

Average Production by Season:
Season
Whole Year     2.395012e+06
Winter         7.182642e+04
```

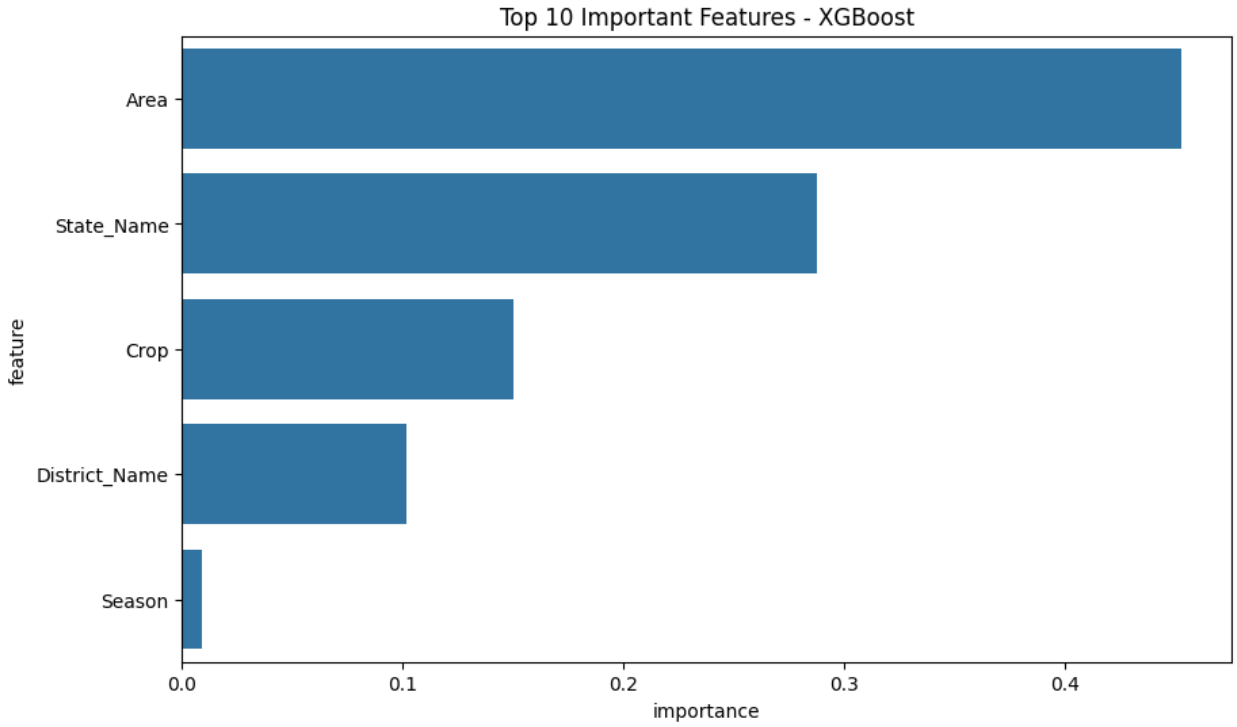
```
Kharif          4.274334e+04
Rabi            3.101100e+04
Autumn          1.306567e+04
Summer          1.152238e+04
Name: Production, dtype: float64
```




```
Training Ensemble...

=== Model Comparison ===
               Model            MAE            MSE           RMSE  R2
Score  \
5      Random Forest  1.924530e+05  4.012875e+13  6.334726e+06
0.900261
4            XGBoost  2.703677e+05  8.348208e+13  9.136853e+06
0.792507
```

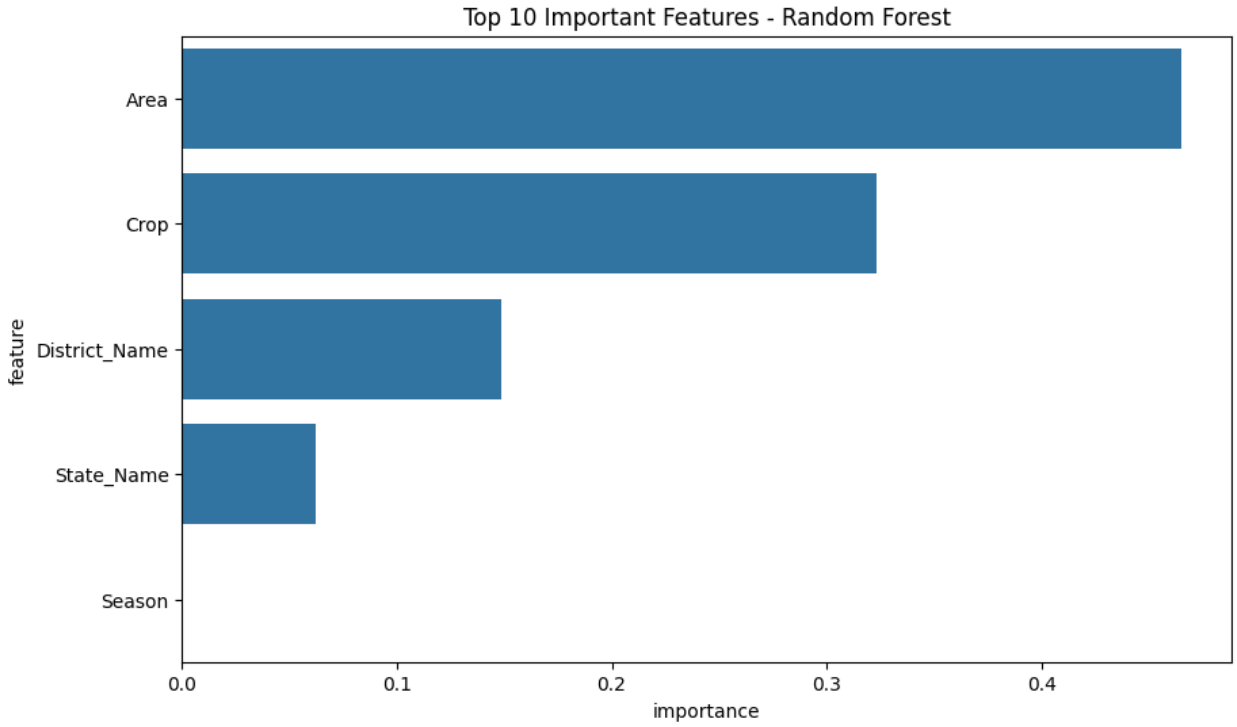
```
6  Gradient Boosting  6.589578e+05  1.058352e+14  1.028762e+07
0.736949
7           Ensemble  1.018743e+06  1.891144e+14  1.375189e+07
0.529961
0  Linear Regression  1.376820e+06  4.010129e+14  2.002531e+07
0.003293
2              Lasso  1.376819e+06  4.010129e+14  2.002531e+07
0.003293
1              Ridge  1.376817e+06  4.010129e+14  2.002531e+07
0.003293
3                SVR  2.806684e+06  4.045000e+14  2.011219e+07 -
0.005374
```

	CV Score Mean	CV Score Std
5	0.839928	0.177884
4	0.815554	0.154431
6	0.723261	0.067799
7	0.550309	0.079999
0	0.002563	0.001025
2	0.002563	0.001025
1	0.002563	0.001025
3	-0.003815	0.002326

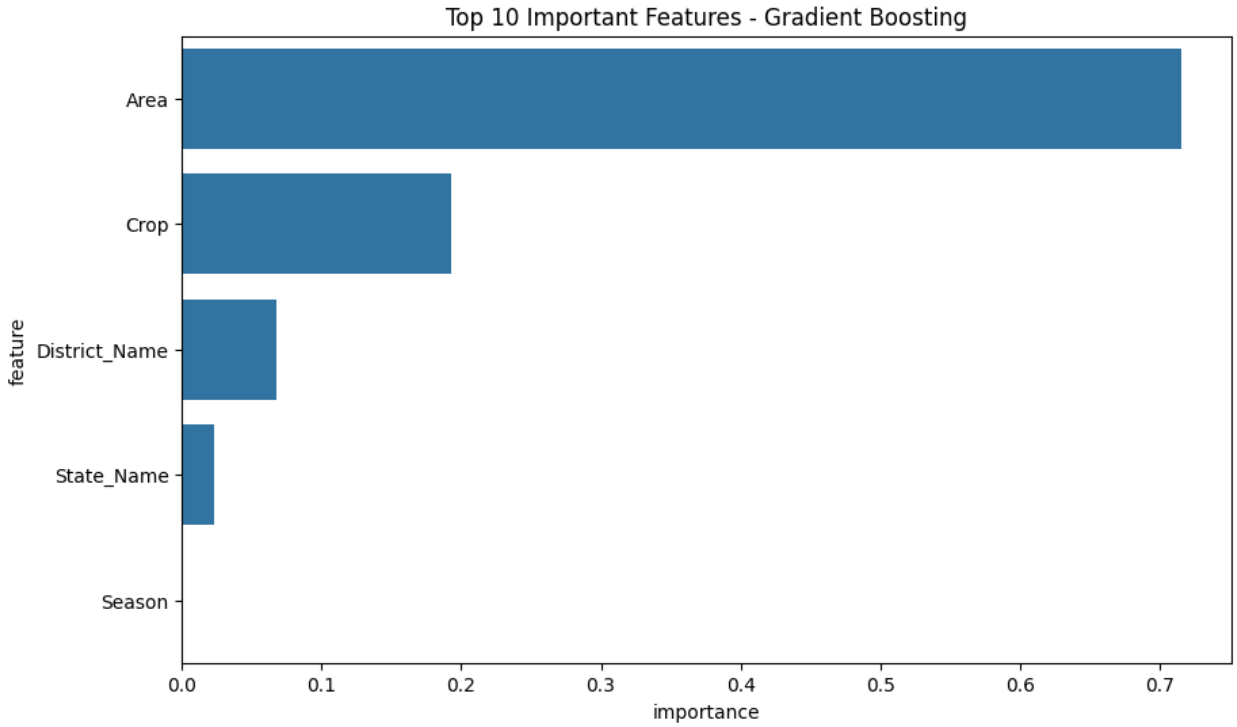



```
Top 10 Important Features - XGBoost:
         feature  importance
```

```
4           Area    0.452667
0     State_Name    0.287232
3           Crop    0.149907
1  District_Name    0.101355
2         Season    0.008840
```




```
Top 10 Important Features - Random Forest:
         feature  importance
4           Area    0.465345
3           Crop    0.323278
1  District_Name    0.148748
0     State_Name    0.062487
2         Season    0.000142
```




```
Top 10 Important Features - Gradient Boosting:
         feature  importance
4           Area    0.716068
3           Crop    0.193132
1  District_Name    0.067422
0     State_Name    0.023367
2         Season    0.000011

Best performing model: Random Forest
Best R2 Score: 0.9003
```